Portfolio Lab Guide — Chapter 2 (Part I): HTML

Internet Programming, I · Department of Software Engineering · Oct 26, 2026

How this lab works

By the end of these 8 labs, every student has a working two-page personal portfolio (`index.html` + `projects.html`) written in plain HTML. Each lab teaches one group of HTML topics from Chapter 2 (Part I) and adds one feature to the same project, so nothing is thrown away.

Rules for students

- Type every line yourself. Do not copy-paste from this guide; typing is how the tags stick.
- Use **your own** name, photo, skills and courses. The examples use a sample student, *Abebe Kebede*; replace his details with yours.
- Save and refresh the browser after every step. If something looks wrong, check the step's *Common mistakes* box before calling the instructor.
- No CSS yet. The page will look plain. Styling comes in the CSS chapter, where we will style this same portfolio.

Every lab follows the same pattern: Goal → Steps (with code) → Checkpoint (what you should see) → Your turn (a small task without code given) → Common mistakes.

Lab	Portfolio feature you add	Time (min)
0. Setup	Project folder, editor, starter files	15
1. Document structure	Valid HTML5 skeleton for `index.html`	20
2. Headings, paragraphs, ` `, `<hr>`	Page outline: name, About, Skills, Projects, Education, Contact	30
3. Formatting, quotes, code	Rich About Me text, favourite quote, "My first program" snippet	35
4. Hyperlinks	Navigation menu, social links, email/phone, CV download, second page	40
5. Images, image map, favicon	Profile photo, project screenshots, clickable "Connect" banner, tab icon	40
6. Lists	Skills (nested), learning roadmap (ordered), tools glossary (description list)	35
7. Tables	Academic record and projects summary table	45

Total is about 4.5 hours. Suggested split: **Lab session A** = Labs 0–4, **Lab session B** = Labs 5–7 plus submission.

Lab 0 — Setup (15 min)

Goal: a project folder that every later lab writes into.

1. Open **VS Code** (any plain-text editor works; never Word or Notepad with rich text).
2. Optional but recommended: install the **Live Server** extension so the browser refreshes on every save.
3. Create this folder structure on your Desktop. Folder and file names are lowercase with no spaces:

```
portfolio/
├── index.html            ← home page (Labs 1–7)
├── projects.html         ← second page (Lab 4 onward)
├── images/
│   ├── profile.jpg       ← your own photo, about 400×400
│   ├── favicon.png       ← 32×32 tab icon
│   ├── connect-banner.png← 600×200 banner for the image map
│   ├── project-calculator.png
│   └── project-library.png
└── files/
    └── cv.pdf            ← your CV (any PDF for now)
```

4. Copy the files from the instructor's **starter pack** into `images/` and `files/`. Replace `profile.jpg` with your own photo when you can.
5. In VS Code choose **File → Open Folder** and open `portfolio`. Keep the browser and editor side by side for the rest of the lab.

Checkpoint: VS Code's Explorer panel shows the tree above.

Common mistakes: saving as `index.html.txt` (turn on *show file extensions*); spaces or capitals in file names, which break links later on Linux servers.

Lab 1 — Document structure (20 min)

Goal: a valid HTML5 page with your name in the browser tab.

1. Open `index.html` and type the basic HTML skeleton below. Two lines are new: `lang="en"` tells browsers and screen readers the page language, and `<meta charset="UTF-8">` lets you type any character (é, አማርኛ) without it turning into symbols.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Abebe Kebede | Portfolio</title>
</head>
<body>
    <h1>Hello, I am Abebe Kebede</h1>
    <p>This is my portfolio website.</p>
</body>
</html>
```

2. Save, then open the file in the browser (double-click it, or right-click → *Open with Live Server*).

3. Right-click the page → **View Page Source**. Then right-click → **Inspect** and expand `<html>`, `<head>` and `<body>` in the Elements panel. In the line *<h1>Hello, I am Abebe Kebede</h1>*, name each part of the element: opening tag, content, closing tag.

Checkpoint: the browser tab reads *Abebe Kebede | Portfolio* (with your name) and the page shows one big heading and one sentence.

Your turn: use the Inspect panel to point at the *attribute* and the *attribute value* in `<html lang="en">`. Tell your neighbour which is which.

Common mistakes: text typed inside `<head>` does not show on the page; only `<body>` content is visible. A missing `</html>` or `</body>` still renders, because browsers forgive errors, but it is still wrong. Close every tag you open.

Lab 2 — Headings, paragraphs, line breaks, rules (30 min)

Goal: the full outline of the home page, using a correct heading hierarchy.

1. Plan before you type. Your page has **one** `<h1>` (your name), one `<h2>` per section, and `<h3>` for items inside a section:

```
h1  Abebe Kebede
    h2  About Me
    h2  Skills
    h2  Projects
        h3  Simple Calculator
        h3  Library Management System
    h2  Education
        h3  BSc in Software Engineering
    h2  Contact
```

2. Replace everything inside `<body>` with the outline below. Write real sentences about yourself in the paragraphs.

```
<body>
    <h1>Abebe Kebede</h1>
    <p>Software Engineering student and aspiring web developer.</p>

    <hr>

    <h2>About Me</h2>
    <p>I am a third-year Software Engineering student. I enjoy building
       websites and solving problems with code.</p>

    <hr>

    <h2>Skills</h2>
    <p>My skills list will go here (Lab 6).</p>

    <hr>

    <h2>Projects</h2>
    <h3>Simple Calculator</h3>
    <p>A calculator that adds, subtracts, multiplies and divides.</p>
```

```
    <h3>Library Management System</h3>
    <p>A console program for borrowing and returning books.</p>

    <hr>

    <h2>Education</h2>
    <h3>BSc in Software Engineering</h3>
    <p>Addis Ababa Science and Technology University, 2023 – present</p>

    <hr>

    <h2>Contact</h2>
    <p>Address:<br>Addis Ababa,<br>Ethiopia</p>
</body>
```

3. Experiment: in the About Me paragraph press Enter five times between two words and add many spaces. Save and refresh. The browser collapses them into one space. This is why `<br>` exists.

Checkpoint: the page shows six sections separated by horizontal lines, and the address prints on three lines.

Your turn: add a fourth project under Projects with its own `<h3>` and `<p>`. Then open the browser's Inspect panel and confirm there is exactly one `<h1>` on the page.

Common mistakes: choosing a heading because of its size (that is CSS's job later; choose by level); skipping levels, such as `<h2>` straight to `<h4>`; using many `<br>` tags to add space between sections instead of separate paragraphs or `<hr>`.

Lab 3 — Text formatting, quotations, code (35 min)

Goal: a richer About Me section, a favourite-quote block and a "My first program" code snippet.

Part A — Formatting the About Me text

1. Rewrite the About Me paragraph so it uses at least five formatting elements. Choose by *meaning*: `<strong>` = important, `<em>` = stressed, `<mark>` = highlighted; `<b>` and `<i>` only change the look.

```
<h2>About Me</h2>
<p>I am a 3<sup>rd</sup> year <strong>Software Engineering</strong> student.
   I <em>really</em> enjoy building websites, and my current goal is to
   <mark>become a full-stack web developer</mark>.</p>
<p>Languages I have used: <b>Python</b>, <b>C++</b> and <i>a little</i>
JavaScript.</p>
<p>Fun fact: I drink 2 litres of H<sub>2</sub>O while coding every day.</p>
```

2. Refresh and check each element. Then right-click → Inspect on the bold words: `<b>` and `<strong>` look the same, but a screen reader stresses only `<strong>`.

Part B — Favourite quote and book

3. Add a new section after About Me:

```
<hr>

<h2>What Inspires Me</h2>
<blockquote>
    “The best way to predict the future is to invent it.” — Alan Kay
</blockquote>
<p>My teacher always says <q>practice beats theory</q>, so I code every day.</p>
<p>Book I am reading: <cite>Clean Code</cite> by Robert C. Martin</p>
```

Part C — "My first program"

4. Under Projects, after the last project, add a short code snippet. `<pre>` keeps spaces and line breaks; `<code>` marks text as code.

```
<h3>My First Program</h3>
<p>The first thing I wrote in Python was <code>print("Hello, World!")</code>.
   Later I wrote this loop:</p>
<pre><code>for i in range(3):
    print("Hello", i)
</code></pre>
```

Checkpoint: a highlighted phrase, a superscript "rd", a subscript "2", an indented quote block, a quote with automatic quotation marks, an italic book title, and a code block that keeps its indentation.

Your turn: add a code snippet in a second language you know (C++ or Java). If your code contains `<` or `>`, write them as `<` and `>`, otherwise the browser treats them as tags. Example: `#include <iostream>`.

Common mistakes: typing quotation marks around `<q>` text (the browser adds them); wrapping a whole paragraph in `<strong>` (emphasis loses meaning when everything is emphasised); writing code in `<pre>` without `<code>`, or the reverse, for multi-line snippets.

Lab 4 — Hyperlinks (40 min)

Goal: a navigation menu, working contact links, a CV download and a second page. You will use all five link types: relative, absolute, email, phone and file download.

Part A — A second page (relative URL)

1. Create `projects.html` with the Lab 1 skeleton. Give it the title `Projects | Abebe Kebede`, an `<h1>My Projects</h1>`, and a link back home:

```
<body>
    <p><a href="index.html">← Back to Home</a></p>
    <h1>My Projects</h1>
    <p>Details of every project I have built. The summary table comes in Lab 7.</p>
</body>
```

2. In `index.html`, at the end of the Projects section, link to the new page:

```
<p><a href="projects.html">See all my projects →</a></p>
```

Part B — Navigation menu (links inside the same page)

3. A link can jump to a spot on the same page. Give each `<h2>` an `id`, then link to it with `#`. Add `id`s like this:

```
<h2 id="about">About Me</h2>
<h2 id="skills">Skills</h2>
<h2 id="projects">Projects</h2>
<h2 id="education">Education</h2>
<h2 id="contact">Contact</h2>
```

4. Under your tagline (before the first `<hr>`), add the menu:

```
<p>
    <a href="#about">About</a> |
    <a href="#skills">Skills</a> |
    <a href="#projects">Projects</a> |
    <a href="#education">Education</a> |
    <a href="#contact">Contact</a> |
    <a href="projects.html">All Projects</a>
</p>
```

Part C — Contact links (absolute, email, phone, download)

5. Replace the Contact paragraph with:

```
<h2 id="contact">Contact</h2>
<p>Address:<br>Addis Ababa,<br>Ethiopia</p>
<p>Email: <a href="mailto:abebe.kebede@example.com">abebe.kebede@example.com</a></p>
<p>Phone: <a href="tel:+251911223344">+251 911 223 344</a></p>
<p>GitHub: <a href="https://github.com/abebekebede">github.com/abebekebede</a></p>
<p>LinkedIn: <a
href="https://www.linkedin.com/in/abebekebede">linkedin.com/in/abebekebede</a></p>
<p><a href="files/cv.pdf" download>Download my CV (PDF)</a></p>
```

Checkpoint: click every link. The menu jumps to each section; *All Projects* opens `projects.html` and *Back to Home* returns; the email link opens a mail app; the CV link downloads the file instead of opening it.

Your turn: in `projects.html`, add a link to the official W3C HTML reference (`https://www.w3.org/`) and a link that jumps straight to the Contact section of the home page. Hint: a page name and an `id` can be combined in one `href`.

Common mistakes: writing `href="www.github.com"` without `https://` (the browser looks for a local file called *www.github.com*); curly quotes `“ ”` pasted from a document instead of straight quotes `" "` (for example *<a href=“about.html">* breaks the link); an `id` with spaces, or two elements with the same `id`.

Lab 5 — Images, image map, favicon (40 min)

Goal: a profile photo, project screenshots, a clickable "Connect with me" banner and an icon in the browser tab.

Part A — Profile photo

1. Directly under `<h1>`, add your photo. The path is relative to `index.html`, so it starts with the folder name.

```
<img src="images/profile.jpg" alt="Portrait of Abebe Kebede smiling"
     title="Abebe Kebede" width="200" height="200">
```

2. Test the `alt` text: change `src` to `images/profil.jpg` (a typo), save and refresh. The browser shows the alt text instead of the image. Fix the typo.

Part B — Project screenshots

3. Add one screenshot under each project heading, in both `index.html` and `projects.html`:

```
<h3>Simple Calculator</h3>
<img src="images/project-calculator.png" alt="Screenshot of the calculator showing 12
+ 30 = 42"
     width="400" height="250">
<p>A calculator that adds, subtracts, multiplies and divides.</p>
```

Part C — "Connect with me" image map

4. The starter file `connect-banner.png` is 600×200 pixels and has three clickable areas: a GitHub rectangle on the left, a LinkedIn circle in the middle and an Email rectangle on the right. At the end of the Contact section, add:

```
<h3>Connect with me</h3>
<img src="images/connect-banner.png" usemap="#connectmap"
     alt="Connect banner: GitHub, LinkedIn, Email" width="600" height="200">

<map name="connectmap">
    <area shape="rect"   coords="20,40,180,160"
href="https://github.com/abebekebede" alt="GitHub">
    <area shape="circle" coords="300,100,70"
href="https://www.linkedin.com/in/abebekebede" alt="LinkedIn">
    <area shape="rect"   coords="420,40,580,160"
href="mailto:abebe.kebede@example.com" alt="Email">
</map>
```

5. Read the numbers: `rect` = x1,y1 (top-left), x2,y2 (bottom-right); `circle` = centre-x, centre-y, radius. Hover over each shape: the pointer becomes a hand only inside the shape.

Part D — Favicon

6. In the `<head>` of **both** pages, under `<title>`, add:

```
<link rel="icon" type="image/png" href="images/favicon.png">
```

Checkpoint: your photo is at the top, each project has a screenshot, the three banner shapes open GitHub, LinkedIn and your mail app, and both browser tabs show the small icon. Favicons are cached, so press Ctrl+F5 if the icon does not appear.

Your turn: make your own favicon at 32×32 (your initials on a coloured square) in Paint or an online editor, save it as `images/favicon.png`, and replace the starter one.

Common mistakes: `usemap="connectmap"` without the `#`; an `alt` that says "image" or "photo" (describe what is *in* it); writing `width` and `height` that do not match the image's proportions, which stretches it; absolute paths such as `C:\Users\...\profile.jpg`, which break as soon as the folder moves.

Lab 6 — Lists (35 min)

Goal: a grouped Skills list, a numbered learning roadmap and a glossary of the tools you use. Each list type fits a different kind of content:

List	Use when	Portfolio section
`<ul>` unordered	order does not matter	Skills
`<ol>` ordered	order or steps matter	Learning roadmap
`<dl>` description	term + its explanation	Tools I use
nested	items have sub-items	Skills grouped by area

Part A — Skills (unordered and nested)

1. Replace the Skills placeholder paragraph. A nested list goes **inside** an `<li>`, before that `</li>` closes.

```
<h2 id="skills">Skills</h2>
<ul>
    <li>Programming Languages
        <ul type="square">
            <li>Python</li>
            <li>C++</li>
            <li>Java</li>
        </ul>
    </li>
    <li>Web Development
        <ul type="circle">
            <li>HTML</li>
            <li>CSS (learning)</li>
            <li>JavaScript (next)</li>
        </ul>
    </li>
    <li>Soft Skills
        <ul>
            <li>Teamwork</li>
            <li>Presenting</li>
        </ul>
    </li>
</ul>
```

Part B — Learning roadmap (ordered)

2. Under Skills, add the order in which you plan to learn. Try each `type` value (`1`, `A`, `a`, `I`, `i`) and keep the one you like.

```
<h3>My Learning Roadmap</h3>
<ol type="1">
    <li>HTML structure and content</li>
    <li>CSS layout and design</li>
    <li>JavaScript and the DOM</li>
    <li>PHP and databases</li>
    <li>Deploy my portfolio online</li>
</ol>
```

Part C — Tools I use (description list)

3. After the roadmap, add:

```
<h3>Tools I Use</h3>
<dl>
    <dt>VS Code</dt>
    <dd>The code editor I use for every lab.</dd>
    <dt>Chrome DevTools</dt>
    <dd>Inspects the HTML of any page and helps me find mistakes.</dd>
    <dt>Git and GitHub</dt>
    <dd>Saves versions of my code and shares my projects online.</dd>
</dl>
```

Checkpoint: three groups of skills with different bullet shapes, a numbered roadmap, and tool names with indented descriptions beneath them.

Your turn: in `projects.html`, under each project, add an ordered list titled *How I built it* with at least 4 steps, and an unordered list of the technologies used.

Common mistakes: putting the nested `<ul>` after `</li>` instead of inside it; text directly inside `<ul>` without an `<li>`; using `<br>` to fake a list. Note: the `type` attribute on `<ul>` is from older HTML and still works, but in the CSS chapter we will replace it with `list-style-type`.

Lab 7 — Tables (45 min)

Goal: an Academic Record table in the Education section that uses caption, header and footer sections, rowspan and colspan, and a projects summary table on `projects.html`.

Part A — Sketch first

1. On paper, draw the grid below. Mark which cells span more than one row (`rowspan`) or column (`colspan`). Tables with merged cells are much easier to code once you have drawn them.

```
+--------+-------------+-------------------------------+--------------+-------+
| Year   | Semester    | Course                        | Credit Hours | Grade |   <-
thead
+--------+-------------+-------------------------------+--------------+-------+
|        | Semester I  | Fundamentals of Programming   | 5            | A     |
|        |             | Applied Mathematics I         | 4            | B+    |
| Year 1 +-------------+-------------------------------+--------------+-------+   <-
tbody
|        | Semester II | Object Oriented Programming   | 5            | A     |
|        |             | Discrete Mathematics          | 4            | B     |
+--------+-------------+-------------------------------+--------------+-------+
| Year 2 | Semester I  | Data Structures & Algorithms  | 5            | A-    |
|        |             | Internet Programming I        | 5            | A     |
+--------+-------------+-------------------------------+--------------+-------+
| Total credit hours (colspan 3)                       | 28           |       |   <-
tfoot
+------------------------------------------------------+--------------+-------+
```

Part B — Code the table

2. In the Education section, after the university paragraph, add the table. Read the comments: a row *skips* any cell that a rowspan from above already covers.

```
<table border="1" cellpadding="8" cellspacing="0">
    <caption>Academic Record — BSc in Software Engineering</caption>
```

```
    <thead>
        <tr>
            <th>Year</th>
            <th>Semester</th>
            <th>Course</th>
            <th>Credit Hours</th>
            <th>Grade</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <th rowspan="4">Year 1</th>
            <td rowspan="2">Semester I</td>
            <td>Fundamentals of Programming</td>
            <td>5</td>
            <td>A</td>
        </tr>
        <tr>
            <!-- Year and Semester are already covered -->
            <td>Applied Mathematics I</td>
            <td>4</td>
            <td>B+</td>
        </tr>
        <tr>
            <!-- Year is still covered -->
            <td rowspan="2">Semester II</td>
            <td>Object Oriented Programming</td>
            <td>5</td>
            <td>A</td>
        </tr>
        <tr>
            <td>Discrete Mathematics</td>
            <td>4</td>
            <td>B</td>
        </tr>
        <tr>
            <th rowspan="2">Year 2</th>
            <td rowspan="2">Semester I</td>
            <td>Data Structures &amp;amp; Algorithms</td>
            <td>5</td>
            <td>A-</td>
        </tr>
        <tr>
            <td>Internet Programming I</td>
            <td>5</td>
            <td>A</td>
        </tr>
    </tbody>
    <tfoot>
        <tr>
            <th colspan="3">Total credit hours</th>
            <th>28</th>
            <th></th>
        </tr>
```

```
    </tfoot>
</table>
```

3. Experiment with the table styling attributes: change `cellpadding` to `2` and `20`, and `cellspacing` to `0` and `10`. Watch what each one changes, then set them back.

Part C — Projects summary table

4. In `projects.html`, under the `<h1>`, build a table on your own with this structure: a caption *Project Summary*; header row **Project | Language | Year | Link**; one row per project, where the Link cell holds an `<a>` to that project's section or GitHub page; a `<tfoot>` row "Total projects" spanning 3 columns, with the count in the last cell.

Checkpoint: the Academic Record has a caption above it, "Year 1" spans 4 rows, each semester spans 2 rows, the footer "Total credit hours" spans 3 columns, and no row has an extra or missing cell (the grid lines all meet).

Your turn: replace the sample courses with your real courses and grades, and update the total.

Common mistakes: counting a rowspan cell again in the next row, which pushes cells to the right; forgetting that `colspan="3"` replaces three cells, so the row needs only 2 more; using a table to lay out the whole page (tables are for data only). Note: `border`, `cellpadding` and `cellspacing` still work but are old HTML; we will move them to CSS next chapter.

Submission, grading and instructor notes

Submit by pushing your finished `portfolio` folder to a public GitHub repository at the end of Lab session B, then send the repository link to the instructor. Do not submit a zip.

Pushing your project to GitHub (15 min)

1. Check that Git is installed: open a terminal (in VS Code: **Terminal → New Terminal**) and run `git --version`. If it is not found, install Git from git-scm.com and restart VS Code.
2. Tell Git who you are (once per computer). Use the same email as your GitHub account:

```
git config --global user.name "Abebe Kebede"
git config --global user.email "abebe.kebede@example.com"
```

3. Sign in at github.com (create a free account if you do not have one). Click **+ → New repository**, name it `portfolio`, set it to **Public**, and leave *Add a README* **unticked** so the repository starts empty. Click **Create repository**.
4. In the VS Code terminal, make sure you are inside the `portfolio` folder (the folder that contains `index.html`), then run:

```
git init
git add .
git commit -m "Chapter 2 Part I: HTML portfolio"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/portfolio.git
git push -u origin main
```

5. The first push asks you to sign in to GitHub; a browser window opens, so approve it there. GitHub no longer accepts your account password in the terminal, so if you are asked for a password, use a personal access token instead.

6. Refresh the repository page on GitHub. You should see `index.html`, `projects.html`, `images/` and `files/` at the top level, not inside another `portfolio` folder.
7. Copy the repository URL (for example `https://github.com/abebekebede/portfolio`) and submit it as the instructor tells you.

Fixing something after you push: edit the file, then run `git add .`, `git commit -m "Describe the fix"` and `git push`. The instructor marks the last commit made before the deadline.

Common mistakes: running `git init` in the wrong folder (e.g. Desktop), which uploads everything on it; ticking *Add a README* when creating the repository, which makes the first push fail (fix with `git pull origin main --allow-unrelated-histories`, then push again); a private repository the instructor cannot open; uploading by drag-and-drop on the website instead of using commit and push.

Submission checklist (tick before your final push)

- ☐ Both pages open with no broken images and no broken links
- ☐ Exactly one `<h1>` per page; no skipped heading levels
- ☐ My own name, photo, skills, courses and contact details (no "Abebe Kebede" left)
- ☐ Every `<img>` has a meaningful `alt`
- ☐ Favicon shows on both pages
- ☐ All three image-map areas work
- ☐ Both tables have a caption, and the Academic Record has `thead`, `tbody`, `tfoot`, a rowspan and a colspan
- ☐ Both pages pass the W3C Markup Validator with no errors except "obsolete attribute" messages for `type`, `border`, `cellpadding` and `cellspacing` (we replace those with CSS next chapter)
- ☐ All changes committed and pushed; the public repository shows `index.html` at the top level, and its link is submitted

Grading rubric (20 marks)

Criterion	Marks	Full marks when
Document structure	2	Valid skeleton on both pages; `lang`, `charset`, `title` present
Headings and text	3	Correct hierarchy; `p`, `br`, `hr` used for their purpose; 5+ formatting elements; quote, cite and code used
Links	4	In-page menu, both pages linked both ways, absolute, `mailto`, `tel` and download links all work
Images and favicon	3	Photo and screenshots with good `alt`; favicon on both pages
Image map	2	Rect and circle areas with correct coordinates and `alt`
Lists	3	`ul`, `ol`, `dl` and a correctly nested list
Tables	3	Academic Record and Project Summary complete, cells line up